

TOPIC

LLVM Pass Transformation Analyzer

Name: Vian Mangal

Register Number: 24BAI0134

Subject: Compiler Design Lab

1. Abstract

This project is a simple educational tool that shows how LLVM optimization passes change a C program. It runs Clang, collects LLVM IR snapshots, removes repeated unchanged output, and produces a short transformation report. This helps students understand the compiler optimization pipeline step by step.

2. Problem Statement

LLVM can print the result of its optimization passes, but the raw output is long and difficult to follow. Students need a small tool that organizes this output and clearly shows which pass changed the program.

3. Motivation

Compiler optimization is often taught using only the original program and the final output. The analyzer exposes the intermediate stages, making the process easier to learn, explain, and demonstrate.

4. Objectives

- Accept one small C source file.
- Run Clang with LLVM pass output enabled.
- Extract the pass name, scope, order, and LLVM IR.
- Remove consecutive unchanged snapshots.
- Generate a manifest, timeline, and numbered IR files.
- Show clear error messages when input or compilation fails.

5. Scope

The first version supports small C programs and a local Clang compiler. It creates reports offline. It does not execute the input program, modify LLVM passes, use AI, require a GPU, or use a web server or database.

6. Background Study

Clang converts C source code into LLVM Intermediate Representation, or LLVM IR. LLVM then runs a sequence of optimization passes. A pass may simplify instructions, remove dead code, improve loops, or clean control flow. LLVM can print IR after each pass, but this output must be organized before it is easy to study.

7. Compiler Design Concepts Involved

- Intermediate Representation: LLVM IR is the program form between source code and machine code.
- Optimization Pass: a compiler stage that analyzes or transforms the program.
- Pass Pipeline: the ordered sequence of compiler passes.
- Function and Module Scope: snapshots are compared only within the same scope.
- Dead Code Elimination and Simplification: common changes visible in the saved IR.

8. Proposed Methodology

- Check that the input is an existing C file and locate Clang.
- Run Clang safely as a subprocess at optimization level O1.
- Parse each IR dump into a structured snapshot.
- Compare snapshots and retain only meaningful changes.
- Write the transformation timeline and LLVM IR files.
- Test the parser, compiler boundary, report output, and command line.

9. System Architecture

C Source -> Clang -> IR Parser -> Change Filter -> Report
--

The complete system is one local Python pipeline. It does not need a database, login, backend service, or internet connection.

10. Technology Stack

- Python 3.10 or newer for the analyzer.
- Clang and LLVM for the real compiler pipeline.
- Command-line interface for simple use.
- JSON, Markdown, and .ll files for the report.
- Python unittest for automated testing.

11. Innovation and Feasibility

The analyzer converts a large expert-oriented compiler dump into a small timeline containing only meaningful changes. The project is practical because it uses Python and Clang, needs no dataset or special hardware, and handles a clearly limited input scope.

12. Initial Prototype

- LLVM snapshot model and pass-dump parser completed.
- Unchanged-snapshot filtering completed.
- Safe Clang discovery and execution boundary completed.
- Twenty-one automated tests currently pass.
- Next step: connect the command line and final report writer.